

Nama : Muhammad Lutfi Febriansyah

NIM : 24060124140197

Lab : D1

TUGAS INDIVIDU PRAKTIKUM GRAFIK DAN TEKNIK INTERAKTIF TEKSTUR & BAYANGAN

Soal 1 :

Analisa dan Jelaskan alur / cara kerja texture yang telah dikerjakan

1. Deklarasi Variabel Global

1.1. Code

```
const float BOX_SIZE = 7.0f;
float _angle = 0;
GLuint _textureId;
```

1.2. Penjelasan

Pada bagian ini program mendeklarasikan variabel global yang digunakan selama program berjalan. `BOX_SIZE` digunakan untuk menentukan ukuran kubus, `_angle` digunakan untuk menyimpan nilai rotasi kubus, sedangkan `_textureId` digunakan untuk menyimpan ID texture OpenGL setelah gambar berhasil dimuat.

2. Fungsi Keyboard

2.1. Code

```
void handleKeypress(unsigned char key, int x, int y)
{
    switch (key) {
        case 27:
            exit(0);
    }
}
```

2.2. Penjelasan

Fungsi ini digunakan untuk mendeteksi input keyboard. Pada program ini tombol ESC digunakan untuk keluar dari aplikasi menggunakan fungsi `exit(0)`.

3. Fungsi Membuat Tekstur

3.1. Code

```
GLuint loadTexture(Image* image) {  
  
    GLuint textureId;  
  
    glGenTextures(1, &textureId);  
  
    glBindTexture(GL_TEXTURE_2D, textureId);  
  
    glTexImage2D(  
        GL_TEXTURE_2D,  
        0,  
        GL_RGB,  
        image->width,  
        image->height,  
        0,  
        GL_RGB,  
        GL_UNSIGNED_BYTE,  
        image->pixels  
    );  
  
    return textureId;  
}
```

3.2. Penjelasan

Fungsi ini bertugas mengubah gambar bitmap menjadi texture OpenGL. Program membuat ID texture menggunakan `glGenTextures()`, sehingga texture siap digunakan pada objek 3D.

4. Fungsi Inisialisasi Rendering

4.1. Code

```
void initRendering() {  
  
    glEnable(GL_DEPTH_TEST);  
    glEnable(GL_LIGHTING);  
    glEnable(GL_LIGHT0);  
    glEnable(GL_NORMALIZE);  
    glEnable(GL_COLOR_MATERIAL);  
  
    Image* image = loadBMP("bg.bmp");  
  
    _textureId = loadTexture(image);  
  
    delete image;  
}
```

```
}
```

4.2. Penjelasan

Fungsi ini digunakan untuk mengaktifkan fitur-fitur penting OpenGL seperti depth test, lighting, light source, normalisasi, dan color material.

5. Fungsi Resize Window

5.1. Code

```
void handleResize(int w, int h) {  
  
    glViewport(0, 0, w, h);  
  
    glMatrixMode(GL_PROJECTION);  
  
    glLoadIdentity();  
  
    gluPerspective(45.0, (float)w / (float)h, 1.0,  
200.0);  
}
```

5.2. Penjelasan

Fungsi ini dijalankan ketika ukuran window berubah. Program mengatur viewport dan perspektif tampilan menggunakan `gluPerspective()` agar objek 3D tetap terlihat proporsional meskipun ukuran window berubah.

6. Fungsi Utama Menggambar Scene

6.1. Code

```
void drawScene()
```

6.2. Penjelasan

Fungsi ini merupakan bagian utama untuk menggambar seluruh objek pada layar, mengatur posisi kamera.

7. Pengaturan Cahaya

7.1. Code

```
GLfloat ambientLight[] = {0.3f, 0.3f, 0.3f, 1.0f};  
GLfloat lightColor[] = {0.7f, 0.7f, 0.7f, 1.0f};  
GLfloat lightPos[] = {-2 * BOX_SIZE, BOX_SIZE, 4 *
```

```
OX_SIZE, 1.0f};
```

7.2. Penjelasan

Bagian ini digunakan untuk menentukan pencahayaan pada scene. Program mengatur cahaya ambient, warna cahaya, dan posisi sumber cahaya sehingga kubus memiliki efek terang dan gelap sesuai arah cahaya.

8. Rotasi Kubus

8.1. Code

```
glRotatef(-_angle, 1.0f, 1.0f, 0.0f);
```

8.2. Penjelasan

Code ini digunakan untuk memutar kubus pada sumbu X dan Y. Nilai `_angle` akan terus berubah sehingga kubus terlihat berputar secara otomatis selama program berjalan.

9. Menggambar Kubus Berwarna

9.1. Code

```
glBegin(GL_QUADS);
```

9.2. Penjelasan

Pada bagian ini program mulai menggambar sisi-sisi kubus menggunakan primitive `GL_QUADS`. Setiap sisi dibuat menggunakan empat vertex (`glVertex3f()`). Program juga memberikan warna pada sisi kubus menggunakan `glColor3f()`.

10. Mengaktifkan Texture

10.1. Code

```
glEnable(GL_TEXTURE_2D);

glBindTexture(GL_TEXTURE_2D, _textureId);

glTexParameteri(
    GL_TEXTURE_2D,
    GL_TEXTURE_MIN_FILTER,
    GL_LINEAR
);
```

```

glTexParameteri(
    GL_TEXTURE_2D,
    GL_TEXTURE_MAG_FILTER,
    GL_LINEAR
);

glColor3f(1.0f, 1.0f, 1.0f);

glBegin(GL_QUADS);

```

10.2. Penjelasan

Bagian ini digunakan untuk mengaktifkan texture mapping dan menghubungkan texture yang sebelumnya telah dibuat ke objek kubus.

11. Pengaturan Kualitas Texture

11.1. Code

```

glEnable(GL_TEXTURE_2D);
glBindTexture(GL_TEXTURE_2D, _textureId);

```

11.2. Penjelasan

Code ini digunakan untuk mengatur kualitas texture agar gambar terlihat lebih halus ketika diperbesar maupun diperkecil.

12. Memetakan Texture ke Kubus

12.1. Code

```

glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
    GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
    GL_LINEAR);

```

12.2. Penjelasan

Fungsi ini digunakan untuk menentukan koordinat texture pada setiap vertex kubus. Dengan koordinat tersebut, gambar bitmap dapat dipetakan secara tepat ke permukaan kubus sehingga texture terlihat menempel dengan benar.

13. Menonaktifkan Texture

13.1. Code

```

glTexCoord2f();

```

13.2. Penjelasan

Setelah proses texture selesai digunakan, texture mapping dinonaktifkan agar tidak mempengaruhi objek lain yang akan digambar berikutnya.

14. Menampilkan Hasil Render

14.1. Code

```
glutSwapBuffers();
```

14.2. Penjelasan

Fungsi ini digunakan untuk menampilkan hasil render ke layar menggunakan sistem double buffering sehingga animasi terlihat lebih halus dan tidak berkedip.

15. Update Animasi

15.1. Code

```
void update(int value)
```

15.2. Penjelasan

Fungsi ini dipanggil setiap 25 milidetik untuk memperbarui nilai rotasi kubus. Nilai `_angle` akan terus bertambah sehingga kubus tampak terus berputar. Setelah itu program memanggil `glutPostRedisplay()` agar scene digambar ulang

16. Main

16.1. Code

```
int main(int argc, char** argv) {  
    glutInit(&argc, argv);  
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB |  
        GLUT_DEPTH);  
    glutInitWindowSize(400, 400);  
    glutCreateWindow("Texture With Image");  
    initRendering();  
    glutDisplayFunc(drawScene);  
}
```

```

        glutKeyboardFunc(handleKeypress);

        glutReshapeFunc(handleResize);

        glutTimerFunc(25, update, 0);

        glutMainLoop();

        return 0;
    }

```

16.2. Penjelasan

Bagian ini merupakan pusat jalannya program. Program melakukan inisialisasi GLUT, membuat window OpenGL, memanggil fungsi inisialisasi rendering, menghubungkan fungsi display, keyboard, resize, dan update, kemudian menjalankan `glutMainLoop()` agar program terus berjalan dan menampilkan animasi kubus 3D secara real-time

17. ImageLoader.cpp

17.1. Code

```

#include <assert.h>
#include <fstream>
#include "imageloader.h"

using namespace std;

Image::Image(char* ps, int w, int h)
    : pixels(ps), width(w), height(h) {
}

Image::~~Image() {
    delete[] pixels;
}

namespace {

    // Konversi 4 byte ke integer (little-endian)
    int toInt(const char* bytes) {
        return (int)(
            ((unsigned char)bytes[3] << 24) |
            ((unsigned char)bytes[2] << 16) |
            ((unsigned char)bytes[1] << 8) |
            (unsigned char)bytes[0]
        );
    };
}

```

```

}

// Konversi 2 byte ke short
short toShort(const char* bytes) {
    return (short)(
        ((unsigned char)bytes[1] << 8) |
        (unsigned char)bytes[0]
    );
}

// Membaca integer dari file
int readInt(ifstream &input) {
    char buffer[4];
    input.read(buffer, 4);
    return toInt(buffer);
}

// Membaca short dari file
short readShort(ifstream &input) {
    char buffer[2];
    input.read(buffer, 2);
    return toShort(buffer);
}

template<class T>
class auto_array {
private:
    T* array;
    mutable bool isReleased;

public:
    explicit auto_array(T* array_ = NULL)
        : array(array_), isReleased(false) {
    }

    auto_array(const auto_array<T> &aarray) {
        array = aarray.array;
        isReleased = aarray.isReleased;
        aarray.isReleased = true;
    }

    ~auto_array() {
        if (!isReleased && array != NULL) {
            delete[] array;
        }
    }

    T* get() const {
        return array;
    }
}

```



```

    T &operator*() const {
        return *array;
    }

    void operator=(const auto_array<T> &aarray)
    {
        if (!isReleased && array != NULL) {
            delete[] array;
        }

        array = aarray.array;
        isReleased = aarray.isReleased;
        aarray.isReleased = true;
    }

    T* operator->() const {
        return array;
    }

    T* release() {
        isReleased = true;
        return array;
    }

    void reset(T* array_ = NULL) {
        if (!isReleased && array != NULL) {
            delete[] array;
        }

        array = array_;
    }

    T* operator+(int i) {
        return array + i;
    }

    T &operator[](int i) {
        return array[i];
    }
};

```

```

Image* loadBMP(const char* filename) {

    ifstream input;
    input.open(filename, ifstream::binary);

    assert(!input.fail() || !"File tidak
ditemukan!");

```

```

char buffer[2];
input.read(buffer, 2);

assert(buffer[0] == 'B' && buffer[1] == 'M' ||
!"Bukan file bitmap!");

input.ignore(8);

int dataOffset = readInt(input);
int headerSize = readInt(input);

int width;
int height;

switch(headerSize) {

case 40:
    width = readInt(input);
    height = readInt(input);

    input.ignore(2);

    assert(readShort(input) == 24 || !"Gambar
tidak 24 bits per pixel!");
    assert(readShort(input) == 0 || !"Gambar
dikompres!");

    break;

case 12:
    width = readShort(input);
    height = readShort(input);

    input.ignore(2);

    assert(readShort(input) == 24 || !"Gambar
tidak 24 bits per pixel!");

    break;

case 64:
    assert(!"Tidak dapat mengambil OS/2 V2
bitmaps");
    break;

case 108:
    assert(!"Tidak dapat mengambil Windows V4
bitmaps");
    break;

```

```

        case 124:
            assert(!"Tidak dapat mengambil Windows V5
bitmaps");
            break;

        default:
            assert(!"Format bitmap ini tidak
diketahui!");
            }

        // Membaca data bitmap
        int bytesPerRow = ((width * 3 + 3) / 4) * 4;
        int size = bytesPerRow * height;

        auto_array<char> pixels(new char[size]);

        input.seekg(dataOffset, ios_base::beg);
        input.read(pixels.get(), size);

        // Menyusun ulang format warna
        auto_array<char> pixels2(new char[width *
height * 3]);

        for (int y = 0; y < height; y++) {
            for (int x = 0; x < width; x++) {
                for (int c = 0; c < 3; c++) {

                    pixels2[3 * (width * y + x) + c] =
pixels[bytesPerRow * y + 3 * x
+ (2 - c)];
                }
            }
        }

        input.close();

        return new Image(pixels2.release(), width,
height);
    }

```

17.2. Penjelasan

`imageloader.cpp` merupakan file pendukung yang digunakan untuk membaca file gambar bitmap (`.bmp`) dan mengubahnya menjadi texture yang dapat digunakan oleh OpenGL. File ini bertugas membuka file gambar, membaca ukuran gambar, membaca data piksel, serta mengonversi format warna agar sesuai dengan format RGB OpenGL.

Pada program `main.cpp`, file ini digunakan melalui fungsi `loadBMP("bg.bmp")` untuk mengambil gambar yang akan dijadikan texture pada kubus. Setelah gambar berhasil dibaca, data gambar dikirim ke fungsi `loadTexture()` untuk diproses menjadi texture OpenGL menggunakan `glTexImage2D()`.

Secara keseluruhan, `imageloader.cpp` berfungsi sebagai penghubung antara file gambar bitmap dengan program utama OpenGL sehingga texture gambar dapat ditampilkan pada permukaan objek 3D.

18. Cara Kerja

Cara kerja texture pada program ini dimulai dengan membaca file gambar bitmap (`bg.bmp`) menggunakan fungsi `loadBMP()` yang terdapat pada `imageloader.cpp`, sedangkan `imageloader.h` digunakan sebagai header untuk deklarasi class `Image` dan fungsi pembaca BMP. File `imageloader.cpp` akan membaca data gambar, ukuran gambar, serta data piksel lalu mengubah format warna menjadi RGB agar sesuai dengan OpenGL.

Setelah gambar berhasil dibaca, data gambar diproses menjadi texture OpenGL menggunakan fungsi `loadTexture()` pada `main.cpp`. Texture kemudian diaktifkan menggunakan `glEnable(GL_TEXTURE_2D)` dan ditempelkan pada permukaan kubus menggunakan `glTexCoord2f()` untuk mengatur posisi gambar pada setiap sisi objek.

Dengan proses texture mapping ini, kubus yang awalnya polos menjadi memiliki tampilan gambar sehingga objek 3D terlihat lebih realistis dan menarik.

Soal 2 :

Buatlah sebuah kubus yang setiap sisinya berisi sebuah gambar.

File ada di github dan folder Tugas

Soal 3 :

Mengubah Object Menjadi Object yang Berbeda

Pada program awal, object yang digunakan adalah Torus dengan kode:

```
glutSolidTorus(20, 40, 20, 10);
```

Pada Main Bayangan sudah ada object yang lain sehingga object dapat diganti dengan object OpenGL lainnya, misalnya Teapot, Sphere, Cube, atau Cone.

Contoh perubahan object menjadi Teapot:

```
void draw() {  
  
    glutSolidTeapot(30.0);  
  
}
```

Contoh Object Lain

Sphere

```
void draw() {  
  
    glutWireSphere(60,10,10);  
  
}
```

Cube

```
void draw() {  
  
    glutSolidCube(40);  
  
}
```

Cone

```
void draw() {  
  
    glutSolidCone(20,50,40,50);  
  
}
```

Penjelasan

Perubahan object dilakukan pada fungsi `draw()` karena fungsi tersebut digunakan untuk menentukan objek 3D yang akan ditampilkan dan dibuat

bayangannya. Program hanya perlu mengaktifkan salah satu object OpenGL, sedangkan object lainnya dapat diberi komentar (//).

Setelah object diganti, program tetap bekerja dengan cara yang sama. Objek asli akan ditampilkan di atas lantai, kemudian fungsi `glShadowProjection()` akan membuat proyeksi bayangan dari object tersebut ke permukaan lantai sesuai posisi sumber cahaya.

Soal 4 :

Analisis dan Jelaskan secara rinci proses membuat bayangannya.

Proses pembuatan bayangan pada program ini dilakukan dengan teknik shadow projection atau proyeksi bayangan. Bayangan dibuat berdasarkan posisi sumber cahaya dan diproyeksikan ke permukaan lantai sehingga object terlihat lebih realistis.

Pertama, program menentukan posisi sumber cahaya menggunakan variabel:

```
float l[] = {0.0, 80.0, 0.0};
```

Variabel ini menentukan letak cahaya pada sumbu X, Y, dan Z. Posisi cahaya akan mempengaruhi arah bayangan object.

Kemudian program menentukan bidang lantai menggunakan variabel:

```
float n[] = {0.0, -40.0, 0.0};  
float e[] = {0.0, -60.0, 0.0};
```

Bidang lantai ini digunakan sebagai tempat jatuhnya bayangan.

Pada fungsi `render()`, program pertama-tama menggambar object asli menggunakan fungsi:

```
draw();
```

Sebelum object ditampilkan, lighting diaktifkan menggunakan:

```
glEnable(GL_LIGHTING);
```

agar object terlihat terkena cahaya dan tampak lebih nyata.

Setelah object asli selesai digambar, program membuat bayangan dengan memanggil fungsi:

```
glShadowProjection(l,e,n);
```

Fungsi ini digunakan untuk membuat proyeksi bayangan dari object ke lantai berdasarkan posisi cahaya.

Selanjutnya object digambar kembali, tetapi kali ini sebagai bayangan. Lighting dimatikan menggunakan:

```
glDisable(GL_LIGHTING);
```

supaya bayangan terlihat gelap seperti bayangan asli.

Warna bayangan diatur menggunakan:

```
glColor3f(0.4,0.4,0.4);
```

sehingga bayangan tampil berwarna abu-abu gelap.

Agar object terlihat bergerak, program memutar object menggunakan:

```
glRotatef(ry,0,1,0);  
glRotatef(rx,1,0,0);
```

Nilai rotasi terus bertambah pada fungsi `idle()`, sehingga object dan bayangannya terlihat berputar secara otomatis. Secara keseluruhan, program membuat bayangan dengan cara menggambar object asli terlebih dahulu, lalu memproyeksikan object tersebut ke bidang lantai menggunakan `glShadowProjection()` sehingga muncul bayangan sesuai arah cahaya.

Soal 5 :

Jelaskan Fungsi dan alur `glShadowProjection`.

Fungsi `glShadowProjection()` digunakan untuk membuat bayangan object pada lantai berdasarkan posisi sumber cahaya. Fungsi ini bekerja dengan membuat matriks proyeksi bayangan, lalu matriks tersebut digunakan untuk mengubah object asli menjadi bayangan.

Fungsi dimulai dengan parameter:

```
void glShadowProjection(float *l, float *e, float *n)
```

Keterangan:

- l digunakan untuk menyimpan posisi sumber cahaya.
- e digunakan untuk menentukan posisi bidang lantai.
- n digunakan sebagai normal atau arah bidang lantai.

Selanjutnya program menghitung nilai:

```
d = n[0]*l[0] + n[1]*l[1] + n[2]*l[2];
```

Nilai d digunakan untuk mengetahui hubungan posisi cahaya terhadap bidang lantai.

Kemudian program menghitung:

```
c = e[0]*n[0] + e[1]*n[1] + e[2]*n[2] - d;
```

Nilai c digunakan sebagai bagian perhitungan untuk membentuk matriks bayangan.

Setelah itu program membuat matriks bayangan pada array:

```
float mat[16];
```

Matriks ini berisi nilai transformasi yang digunakan untuk mengubah object menjadi bayangan sesuai arah cahaya dan posisi lantai.

Contohnya:

```
mat[0] = l[0]*n[0]+c;
```

dan seterusnya hingga seluruh isi matriks selesai dihitung.

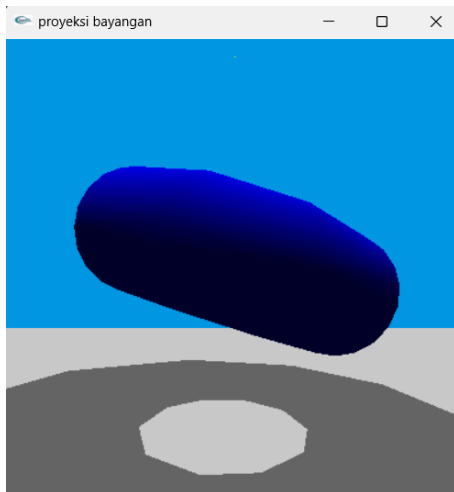
Setelah matriks selesai dibuat, program menggunakan:

```
glMultMatrixf(mat);
```

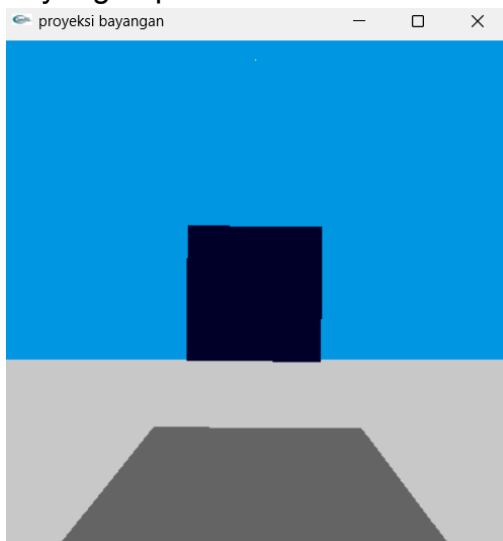
Fungsi ini digunakan untuk menerapkan matriks bayangan ke object. Akibatnya, object yang digambar setelah fungsi ini akan berubah menjadi proyeksi bayangan pada lantai.

Contoh Bayangan yang dihasilkan pada Objek :

Bayangan pada Torus



Bayangan pada Kubus :



Secara sederhana, alur kerja `glShadowProjection()` adalah:

1. Mengambil posisi sumber cahaya.
2. Mengambil posisi bidang lantai.
3. Menghitung matriks bayangan.
4. Menerapkan matriks ke object.
5. Object berubah menjadi bayangan sesuai arah cahaya.

Dengan fungsi ini, program dapat menampilkan efek bayangan sederhana pada object 3D sehingga tampilan terlihat lebih realistis.